Predictive Content Placement in CCN-based IoT Networks via Digital Twin and Cloud-Edge Collaboration

Soulaymane KEBLI & Zoé POIRIER
Supervised by Mme Ghada JABER

***Abstract*—This paper investigates the integration of Digital Twin (DT) technology to enhance Content-Centric Networking (CCN) in Internet of Things (IoT) environments. Traditional reactive caching strategies, such as LRU or LCE, often prove inefficient in managing the massive and dynamic traffic generated by modern IoT devices due to strict hardware constraints. We propose a hierarchical Cloud-Edge architecture where a centralized Digital Twin identifies global popularity trends and coordinates with local "Super Nodes" elected based on network centrality. These edge agents execute a proactive Greedy-based allocation algorithm optimized by a multi-criteria profitability score that simultaneously considers predicted popularity, content size, and potential latency gain. Through extensive simulations conducted in the ndnSIM framework using a specialized pre-fetching mechanism at $T = 0.1s$, we demonstrate that our proactive approach significantly outperforms legacy models. Our results show a 75% reduction in retrieval latency for prioritized content while preventing core network congestion and optimizing global resource utilization across constrained IoT clusters.**

***Index Terms*—Digital Twin (DT), Content-Centric Networking (CCN), IoT, Greedy Algorithm, ndnSIM, Cloud-Edge Collaboration, Predictive Caching.**

I. Introduction

The exponential growth of IoT devices challenges traditional IP architectures, which were originally designed for host-to-host communication. Content-Centric Networking (CCN) addresses these issues by focusing on named data, allowing intermediate routers to cache content. However, IoT routers are often resource-constrained, making the management of the Content Store (CS) a critical bottleneck.

Standard caching policies like Least Recently Used (LRU) or Leave Copy Everywhere (LCE) are inherently reactive: they only store data after a request has already traversed the network. To achieve proactive management, we introduce the Digital Twin Network (DTN). A DT serves as a virtual replica that collects real-time traffic data and performs "What-if" analysis to optimize the physical network. This paper focuses on a DT-driven caching strategy that anticipates content demands using a hierarchical Cloud-Edge model and a Greedy-based selection process.

II. Related Work

Digital Twin Networks are gaining traction as a data-driven performance modeling tool. According to Wu et al. [1], DTNs provide a safe environment to test configurations using Machine Learning (ML). While general frameworks exist, specialized applications for CCN-IoT are still emerging.

Articles in the field highlight the "data gap" between simulation and real production data [1]. Almasan et al. [2] explore the potential of Graph Neural Networks (GNN) to model network topology. Other researchers, such as Yi et al. [3], emphasize the importance of Cloud-Edge-End (E-E-C) collaboration to distribute intelligence. Our work builds upon these foundations by proposing a lightweight but mathematically sound Greedy allocation specifically designed for "Super Nodes" at the network edge, bridging the gap between computational complexity and real-time responsiveness.

III. System Architecture: Cloud-Edge Collaboration

To reconcile high computational power with low-latency requirements, our architecture follows a dual-plane model inspired by real-time IoT constraints.

A. Centralized Digital Twin (Cloud Layer)

The centralized DT layer maintains a global view of the entire network topology. It periodically gathers telemetry data, such as traffic volumes and link utilization. Its primary role is to execute heavy Deep Learning models to predict global content popularity trends ($\hat{p}_i$). By analyzing large-scale patterns (e.g., a viral video release), the Cloud DT determines which contents should be prioritized across different zones. This "heavy-lifting" is performed in the Cloud because IoT nodes lack the GPU/RAM capacity for complex training.

B. Edge Agents and Super Nodes

We designate specific routers as "Super Nodes" based on their centrality within the topology.

- **Election Criteria:** We select nodes combining high Degree Centrality (those with the most direct neighbors) and Betweenness Centrality (those frequently traversed for shortest paths).

- **Local Intelligence:** Each Super Node hosts a lightweight AI Agent. This agent receives global trends from the Cloud and applies local context (e.g., specific requests from Zone A vs Zone B) to make final caching decisions. This decentralized execution ensures that the content is cached at the edge, closest to the end-users.

IV. Proposed Solution: Profitability-based Caching

Our solution transforms the Super Node from a passive forwarder into a proactive decision point.

A. Data Modeling: The VideoItem Structure

In our implementation, the Digital Twin treats content as a "Rich Object" rather than raw packets. As defined in our simulation code (*simulation.cc*), each *VideoItem* contains:

- **ID:** The unique content name or identifier.
- **Size (S_i):** The storage footprint in memory blocks (e.g., 2 to 8 blocks).
- **Popularity ($\hat{p}_i$):** The predicted probability of being requested within a window T.
- **Hit Value (G_i):** The gain (latency reduction) if served locally.

B. The Profitability Score

To maximize the utility of the limited Content Store, the DT calculates a *Profitability Score* ($Score_i$):

$$Score_i = \frac{\hat{p}_i \times G_i}{S_i} \tag{1}$$

This formula ensures that small, highly demanded files are prioritized over large, niche content. By maximizing the score, we maximize the number of expected "Hits" per unit of storage.

C. Greedy Allocation Algorithm

The Super Node fills its cache using an iterative Greedy heuristic. This approach solves the 0/1 Knapsack problem inherent to caching under capacity constraints. The algorithm sorts the catalog by score and admits items until the capacity C is reached.

Algorithm 1: Proactive Cache Allocation (Greedy)

Input : Catalogue of items V, Cache Capacity C
Output: Pre-load List P

```
Sort V in descending order of Score_i;
UsedSpace ← 0; P ← ∅;
foreach item v ∈ V do
    if UsedSpace + v.taille ≤ C then
        Add v.id to P;
        UsedSpace ← UsedSpace + v.taille;
        Log: [INFO] Insertion Video ID: v.id (OK);
    else
        Log: [SKIP] Overflow for Video ID: v.id
          (Rejected);
```

V. Implementation and Performance Evaluation

A. Simulation Environment: ndnSIM

We implemented our strategy in **ndnSIM** using a star topology consisting of 3 consumers, 1 Super Node (Router), and 1 remote Producer. The link parameters were set to $10Mbps$ and $10ms$ delay. To simulate a realistic constraint, we strictly limited the Content Store: `ndnHelper.setCsSize(10)`. Given that videos vary between 2 and 8 blocks, optimization is vital.

B. The Pre-fetching Mechanism (Push Logic)

Traditional CCN caches fill reactively (LRU). To implement the DT's "Push" decision, we used a **Pre-fetching** protocol:

1) **Phase 0 (Initialization):** The DT executes the Greedy algorithm in C++ before the network starts.
2) **Phase 1 (Pre-loading):** At $T = 0.1s$, "Artificial Consumers" are triggered to request only the videos in the list P.
3) **Phase 2 (Warming):** These packets are intercepted and stored in the Super Node's CS. We stop the artificial requests at $T = 0.5s$.
4) **Phase 3 (Real Traffic):** Real user traffic begins at $T = 2.0s$.

C. Results Analysis and Scenarios

We analyzed two distinct scenarios to validate the efficiency of the scoring system:

1) Scenario A (Alice - Predictive Success): Alice requests Video 1 ($S_1 = 2$, $\hat{p}_1 = 0.9$). Since $Score_1 = 4.5$, the video was pre-cached by the agent.

- **Result:** Immediate Cache Hit at the Super Node.
- **Performance:** Latency is reduced by **75%**. No traffic is generated toward the core network, saving significant bandwidth.

2) Scenario B (Bob - Controlled Miss): Bob requests Video 2 ($S_2 = 8$, $\hat{p}_2 = 0.1$). Because of its large size and low profitability ($Score_2 = 0.12$), the algorithm rejected it during the Greedy phase to preserve space for smaller, more frequent requests.

- **Result:** Cache Miss. The request is forwarded to the remote producer.
- **Conclusion:** While Bob experiences higher latency, the system's global utility is maximized by serving a larger number of users with high-priority content.

VI. Conclusion

This paper demonstrated a Digital Twin-assisted caching strategy for CCN-IoT. By using a hierarchical Cloud-Edge architecture and a profitability-based Greedy algorithm, we moved from reactive to proactive network management. Simulation results confirm that selecting content based on a "Value-per-Size" ratio is essential for resource-constrained environments. Future work will investigate the use of Graph Neural Networks (GNN) for dynamic Super Node election.

References

[1] Wu et al., "Digital Twin for Networking: A Data-Driven Performance Modeling Perspective," IEEE, 2024.

[2] Almasan et al., "Network Digital Twin: Context, Enabling Technologies, and Opportunities," IEEE, 2022.

[3] Yi et al., "DT-Driven Integrated Caching and Routing for Content Delivery," ScienceDirect, 2024.